

Game Client Programmer Portfolio

UE5 Action Combat Architecture + DirectX Gameplay / Rendering Systems

Primary Focus

Action Combat

Engine / API

UE5 · DX11 · DX9

Portfolio Scope

4 Projects

UE5 Project Video

<https://youtu.be/WXY8d4P0Uec>

DX Projects Video

<https://youtu.be/MyqkYV1b1fc>

Portfolio Direction

이 포트폴리오는 로우 레벨 구현 경험과 엔진 기반 시스템 설계 경험을 함께 보여주는 것을 목표로 합니다. DirectX 프로젝트에서는 렌더링/엔진 구조를 직접 다루며 카메라, 충돌, 파티클, 미니맵 등 게임 시스템을 구현했습니다. UE5 프로젝트에서는 엔진 클래스를 상속/확장해 액션 전투 구조를 직접 설계하고, 유지보수성과 확장성을 고려해 리팩토링했습니다. LLM/Agent는 반복 작업과 로직 점검을 빠르게 처리하는 보조 도구로 활용했으며, 최종 판단과 적용은 직접 검수했습니다.

Portfolio Sections

Profile / Skills



UE5 Combat System



Action Combat Systems



DirectX Projects



Gameplay / Rendering Systems

Profile / Skills

전투 시스템 중심의 게임 클라이언트 프로그래머

Profile

전투 시스템 중심의 Gameplay Programming에 관심이 있으며, UE5 C++ 프로젝트와 DirectX 개인/팀 프로젝트를 통해 전투, 카메라, UI, 렌더링 시스템을 구현했습니다.

Focus Area

Action Combat · Gameplay Programming · Camera Systems · Rendering Systems · Tools for Feedback

Strength

문제를 시스템 단위로 분해하고, 플레이 감각에 필요한 기능을 상태, 정책, 인터페이스, Subsystem으로 구조화합니다. DirectX 프로젝트에서는 수학/좌표계 기반 구현과 렌더링 최적화 포인트를 직접 다뤘습니다.

Skills

Language

C++

Engine / API

Unreal Engine 5

DirectX11 / DirectX9

Systems

Action FSM

Combat Systems

OBB Collision

Particle Instancing

Lock-On Camera

Projects Overview

Unreal 메인 프로젝트와 DirectX 기반 프로젝트 요약

Sekiro-like Combat System

UE5 C++ Personal Project · 60 Days

Action FSM / Defense & Parry / Boss FSM / Common Attack Trace / Execution System

Demon Slayer Recreation

DirectX11 Team Project · 6 Members · 60 Days

Combat Camera / Awakening Cutscene / Minimap UI

Thymesia Recreation

DirectX11 Personal Project · 43 Days

Parry System / Particle Instancing / OBB Weapon Hit Detection

IRA Recreation

DirectX9 Team Project · 4 Members · 30 Days

Time Stop Skill / Player Combat / Camera System

Part 1. Unreal - Sekiro-like Combat System

UE5 C++ 기반 3인칭 액션 보스전 포트폴리오

System Focus

- 실제 조작감을 고려해 입력, 캔슬, 패링, 회피, 점프, 스킬 흐름을 중앙 액션 파이프라인으로 정리했습니다.
- 보스 전투는 상태 전환, 공격 선택, 몽타주 Notify 기반 링크를 통해 상황에 맞는 공방 흐름을 구성했습니다.
- AttackTrace, HitReaction, Execution, VFX/TimeEffect를 인터페이스와 컴포넌트 중심으로 분리해 재사용성을 높였습니다.

Captured from packaged Shipping build

Tutorial + one boss fight · UE5 C++



Gameplay screenshot used as the UE5 project visual reference

Project Overview

전투 시스템을 설명하기 위한 5개 핵심 챕터와 부록 구성

01 Player Action FSM + Cancel / Buffer

입력을 즉시 실행하지 않고 요청으로 변환한 뒤, 상태와 캔슬 정책을 통과시킵니다.

02 Defense / Parry System

우클릭 하나를 탭, 홀드, 체인 패리, 공중 패리, 미키리 준비로 해석합니다.

03 Boss Combat FSM + Attack Link

거리, 페이즈, 쿨타임, 확률을 기반으로 공격을 고르고 Notify에서 다음 공격을 링크합니다.

04 Common Attack Trace + Hit Reaction

플레이어, 보스, NPC가 같은 공격 요청과 피격 인터페이스를 공유합니다.

05 Reusable Execution System

처형하는 쪽과 당하는 쪽을 인터페이스로 분리해 보스와 NPC가 다른 결과를 처리합니다.

Appendix Direction

전투 / 판정

Special Counter

연출 / 피드백

VFX · TimeEffect · BGM

플레이 보조

Lock-On Camera · Tutorial · Potion · Monster HUD · Niagara Trail · Foot IK

01. Player Action FSM / Cancel Buffer

입력 요청, 캔슬 정책, 입력 버퍼를 중앙 파이프라인으로 관리

Problem

액션이 늘어날수록 입력, 캔슬, 후속 버퍼가 함수 곳곳에 흩어져 조작감 튜닝이 어려웠습니다.

Solution

모든 입력을 ActionRequest로 변환하고, 현재 상태와 CancelRule을 통과한 요청만 실행하게 했습니다.

Result

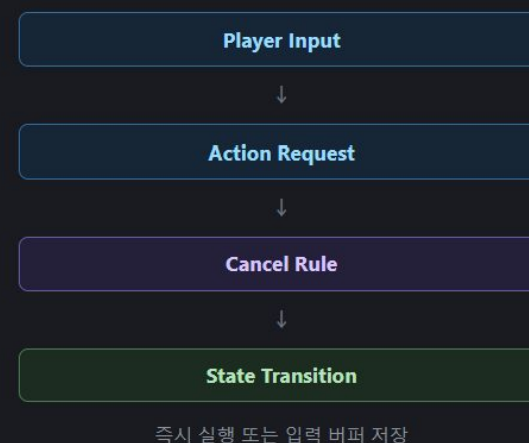
공격, 회피, 점프, 패링 후속 행동을 같은 흐름에서 처리해 액션 추가와 튜닝 부담을 줄였습니다.

01

Player Action FSM Cancel Buffer

입력을 바로 실행하지 않고 ActionRequest로 변환한 뒤, 현재 액션과 캔슬 정책을 검사해서 실행하거나 버퍼에 저장한다.

Input -> Request
Request -> CancelRule
CancelRule -> StateTransition



```
PlayerActionPipeline.cpp · UE5 · C++

// 입력은 바로 실행하지 않고 ActionRequest로 변환한다.
void AJunPlayer::RequestAction(EPlayerAction NextAction)
{
    FActionRequest Request;
    Request.From = CurrentAction;
    Request.To = NextAction;

    // 현재 상태에서 바로 시작할 수 있으면 즉시 실행한다.
    if (CanStartImmediately(Request))
    {
        ProcessAction(Request);
        return;
    }

    // 후딜 캔슬 타이밍이 아직 열리지 않았다면 입력을 저장한다.
    if (CanBuffer(Request))
    {
        PendingAction = Request;
    }
}

void AJunPlayer::TryProcessPendingAction()
{
    if (!PendingAction.IsSet())
        return;

    // FromAction, ToAction에 맞는 캔슬 규칙을 찾는다.
    FCancelRule Rule = FindCancelRule(PendingAction);

    // OpenTime 이전이면 아직 캔슬 불가.
    if (ElapsedTime < Rule.OpenTime)
        return;

    // 기존 몽타주를 부드럽게 빼고 다음 액션을 실행한다.
    StopCurrentMontage(Rule.BlendOutTime);
    ProcessAction(PendingAction);
    PendingAction.Reset();
}
```

02. Defense / Parry System

우클릭 입력을 현재 상태와 타이밍에 따라 패링, 가드, 체인 패리로 해석

Problem

같은 우클릭 입력이 상황에 따라 패링, 가드, 체인 패리, 피격 캔슬로 달라져 입력 해석이 꼬이기 쉬웠습니다.

Solution

DefenseComponent가 입력 유지 시간, 현재 상태, 패링 윈도우, 공격 방향과 타입을 함께 판단합니다.

Result

탭/홀드/체인/공중 방어가 안정적으로 분기되어 타이밍 기반 공방 흐름을 만들 수 있었습니다.

02 Defense Parry System

같은 우클릭 입력을 현재 상태에 따라 패링, 체인 패리, 가드 홀드, 피격 캔슬로 해석한다.

DefenseInput -> StateCheck
AttackHit -> ResolveDefense
Result -> Parry or Guard or Hit

```

graph TD
    A[Defense Input] --> B[State Check]
    B --> C[Parry]
    B --> D[Guard]
    C --> E[Reaction]
    D --> E
    E --- F[탭/홀드/피격 상태에 따라 분기]
    
```

```

PlayerDefenseSystem.cpp · UE5 · C++

void UDefenseComponent::HandleDefenseStarted()
{
    if (Player->IsParrySuccess())
    {
        // 패링 성공 중 재입력: 탭은 체인 패리, 홀드는 가드 전환.
        RefreshChainParryWindow();
        TryConvertToGuardHold();
        return;
    }

    if (Player->IsHitReacting())
    {
        // 피격 중에도 정식 캔슬 타이밍이면 방어로 전환한다.
        Player->TryCancelHitReactIntoDefense();
        return;
    }

    // 일반 입력은 GuardStart를 재생하며 패링 윈도우를 연다.
    StartGuardStart();
    OpenParryWindow();
}

EDefenseResult UDefenseComponent::ResolveDefense(const FAttackData& Attack)
{
    // 공격자를 바라보는 각도 안에 있어야 방어 판정 가능.
    if (!IsFacingAttacker(Attack.Attacker))
        return EDefenseResult::Hit;

    // 완벽 패리 윈도우가 일반 패리보다 우선한다.
    if (IsPerfectParryWindowOpen())
        return EDefenseResult::PerfectParry;

    if (IsParryWindowOpen())
        return EDefenseResult::NormalParry;

    if (CanGuard(Attack))
        return EDefenseResult::GuardBlock;

    return EDefenseResult::Hit;
}
    
```

03. Boss FSM / Attack Link

상태 전환, 공격 후보 선정, 몽타주 Notify 기반 링크로 보스 전투 흐름 구성

Problem

보스를 고정 콤보나 단순 랜덤으로 만들면 거리, 페이스, 쿨타임에 맞춘 공방 흐름을 만들기 어려웠습니다.

Solution

Combat FSM과 AttackSelectionStrategy로 후보를 필터링하고, Notify 시점에서 다음 공격을 링크했습니다.

Result

보스가 접근, 재배치, 공격, 연계를 상황에 맞게 선택해 전투가 덜 반복적으로 느껴지게 했습니다.

03

Boss FSM Attack Link

거리, 페이스, 쿨타임, 확률을 기반으로 공격을 선택하고, 몽타주 Notify에서 다음 공격으로 자연스럽게 링크한다.

StateRequest -> Transition
AttackSelect -> Montage
Notify -> LinkAttack

Boss State

↓

Attack Selection

↓

Attack Montage

↓

Link Notify

거리 조건 실패 시 접근 후 재공격

BossCombatFSM.cpp · UE5 · C++

```
void AJunBoss::RequestCombatState(EBossState NextState)
{
    // 모든 전투 상태 전환은 하나의 진입점에서 검증한다.
    if (!CanTransition(CurrentState, NextState))
        return;

    ExitState(CurrentState);
    CurrentState = NextState;
    EnterState(CurrentState);
}

FBossAttack AJunBoss::SelectNextAttack()
{
    TArray<FBossAttack> Candidates = GetAllAttacks();

    // 거리, 페이스, 쿨타임으로 사용 가능한 공격만 남긴다.
    Candidates = FilterByDistance(Candidates, TargetDistance);
    Candidates = FilterByPhase(Candidates, CurrentPhase);
    Candidates = FilterByCooldown(Candidates);

    // 후보가 없다면 공격 대신 접근 행동으로 전환한다.
    if (Candidates.IsEmpty())
        return MakeApproachAction();

    return WeightedRandom(Candidates);
}

void AJunBoss::OnAttackLinkNotify(const FAttackLinkData& LinkData)
{
    // 몽타주 Notify 시점에서 다음 공격 링크 여부를 결정한다.
    if (!ChancePassed(LinkData.TriggerChance))
        return;

    FBossAttack NextAttack = SelectLinkCandidate(LinkData);

    if (!IsInRange(NextAttack))
    {
        // 사거리 밖이면 먼저 거리를 맞춘 뒤 공격한다.
        MoveToRangeThenAttack(NextAttack);
        return;
    }

    BlendToAttackMontage(NextAttack);
}
```


04. Common Attack Trace / Hit Reaction

공격자는 공통 HitRequest만 만들고, 피격자는 인터페이스로 자기 반응을 선택

Problem

플레이어, 보스, NPC, 투사체 공격을 따로 처리하면 공격 타입과 피격 리액션 코드가 계속 중복됐습니다.

Solution

공격자는 공통 HitRequest를 만들고, 피격자는 인터페이스로 받아 자신의 상태에 맞게 해석하게 했습니다.

Result

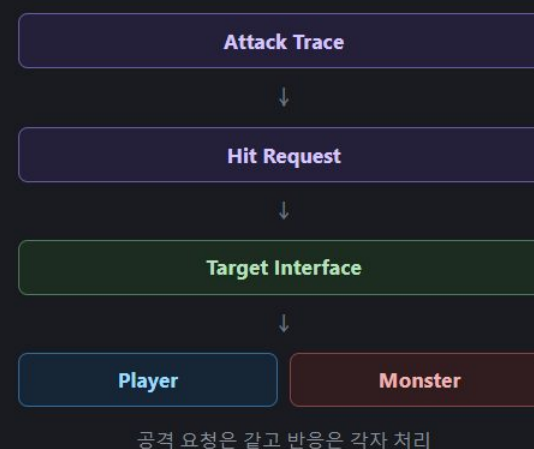
새 공격이나 새 몬스터도 같은 판정 파이프라인에 연결할 수 있어 재사용성을 높였습니다.

04

Common Attack Trace Hit Reaction

공격자는 공통 HitRequest만 만들고, 피격자는 인터페이스를 통해 각자 패링, 가드, 슈퍼아머, 피격 리액션을 해석한다.

AttackTrace -> HitRequest
Interface -> TargetReaction
SuperArmor -> PhysicalHit



CommonCombatHit.cpp · UE5 · C++

```

void UAttackTraceNotify::OnHit(AActor* Target)
{
    // 공격자는 대상 클래스를 모르고 공통 HitRequest만 만든다.
    FAttackHitRequest Request;
    Request.Attacker = Owner;
    Request.Damage = Damage;
    Request.PostureDamage = PostureDamage;
    Request.HitReactType = HitReactType;
    Request.DefenseRule = DefenseRule;

    if (IAttackTarget* HitTarget = Cast<IAttackTarget>(Target))
    {
        // 피격자는 자신의 상태에 맞게 패링, 가드, 피격을 해석한다.
        HitTarget->ReceiveAttack(Request);
    }
}

void AJunMonster::ReceiveAttack(const FAttackHitRequest& Request)
{
    // 방어 계열 결과가 먼저 처리되면 데미지/피격 리액션은 생략된다.
    if (TryParry(Request))
        return;

    if (TryGuard(Request))
        return;

    ApplyDamage(Request.Damage);

    if (HasSuperArmor())
    {
        // 슈퍼아머 중에는 몽타주 전환 대신 피지컬 히트만 적용한다.
        ApplyPhysicalHit(Request);
        return;
    }

    PlayHitReactMontage(Request.HitReactType);
    ApplyPhysicalHit(Request);
}
  
```

05. Reusable Execution System

처형 실행자와 처형 대상의 결과 처리를 인터페이스로 분리

Problem

처형을 보스 전용으로 만들면 TutorialNPC나 일반 몬스터에 재사용하기 어렵고 의존성이 커졌습니다.

Solution

플레이어는 처형 실행만 담당하고, 대상은 ExecutionTarget 인터페이스로 자기 처형 결과를 처리합니다.

Result

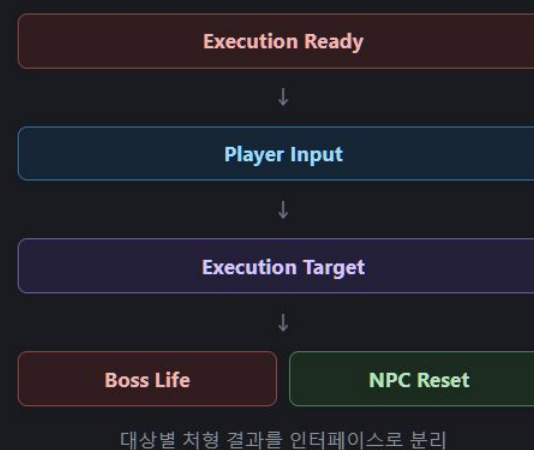
같은 처형 입력을 공유하면서도 보스는 Life 감소, NPC는 회복/Task 완료처럼 다른 결과를 낼 수 있습니다.

05

Reusable Execution System

플레이어는 실행자 역할만 담당하고, 보스와 NPC는 ExecutionTarget 인터페이스를 통해 각자 처형 결과를 처리한다.

PostureBreak -> Ready
Input -> ExecutionTarget
Result -> Boss or NPC Rule



ExecutionSystem.cpp · UE5 · C++

```

bool UExecutionComponent::TryStartExecution()
{
    // 처형 가능 대상을 찾고 인터페이스로 검증한다.
    IExecutionTarget* Target = FindExecutionTarget();

    if (!Target || !Target->CanBeExecutedBy(Player))
        return false;

    // 플레이어의 현재 액션을 끊고 처형 액션으로 전환한다.
    CancelCurrentPlayerAction();
    PlayPlayerExecutionMontage(Target);

    // 대상은 자신의 클래스에 맞는 처형 당하는 흐름을 실행한다.
    Target->BeginExecutedBy(Player);
    return true;
}

void AJunBoss::BeginExecutedBy(AJunPlayer* Executor)
{
    // 보스는 처형 몽타주, Life 감소, 페이지 전환을 처리한다.
    StopAllAttackTraces();
    PlayVictimExecutionMontage(Executor);
    ConsumeBossLife();
    UpdateBossHUD();
}

void AJunTutorialNPC::BeginExecutedBy(AJunPlayer* Executor)
{
    // 튜토리얼 NPC는 죽지 않고 다음 학습 단계로 복구된다.
    PlayVictimExecutionMontage(Executor);
    RestoreHealthAndPosture();
    CompleteTutorialExecutionTask();
}
  
```

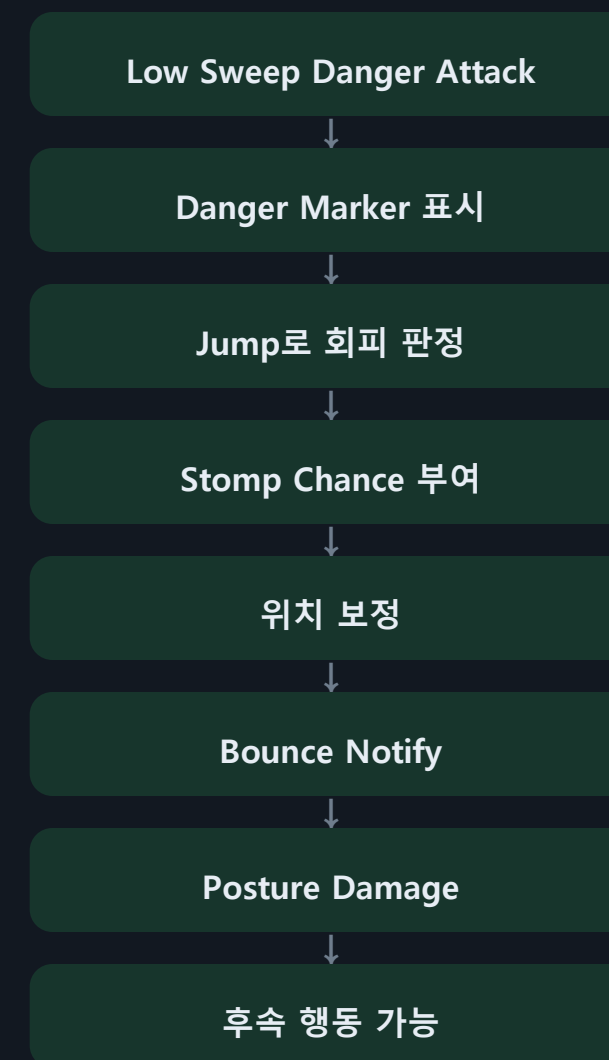
Appendix A. Special Counter System

Mikiri와 Jump Counter는 공격자와 대응자의 세트 플레이로 구성

Mikiri Counter



Jump Counter



위험공격은 일반 방어나 무적 회피로 끝내지 않고, 공격 타입에 맞는 대응을 요구하도록 분리했습니다. Mikiri는 공격자의 가능 구간과 플레이어의 입력 윈도우가 동시에 맞아야 성공하며, Jump Counter는 하단 공격을 점프로 피한 뒤 밟기 후속 행동으로 이어집니다. 보스와 TutorialNPC가 같은 인터페이스 기반 대응 구조를 재사용합니다.

Appendix B. Supporting Systems

본문 5개 챕터 밖에 있는 구현 기능들

VFX / SFX Subsystem

Parry, GuardHit, GuardBreak, Blood, HitStop 요청을 캐릭터 코드에서 분리하고 Subsystem이 Niagara와 사운드를 처리합니다.

Time Effect Subsystem

HitStop과 SlowMotion을 전역/부분 타임 효과로 관리해 처형, 피격, 공격 타격감을 독립적으로 조정합니다.

Lock-On Camera

거리 기반 Pitch와 SpringArm Length, 히스테리시스, 처형/대화/죽음 전용 카메라 값을 별도로 두었습니다.

Tutorial NPC / Task

대화, 안전 이동, 단계별 학습 Task, 더미 모드와 공격 모드를 가진 튜토리얼 NPC를 구현했습니다.

Potion System

충전형 물약, 점진 회복, 무기 숨김/복구, 튜토리얼 회복 완료 이벤트를 PotionComponent로 분리했습니다.

BGM Manager

맵 BGM, 전투 BGM, 처형/죽음 특수 상황의 볼륨 덕킹과 페이드 전환을 관리합니다.

Monster HUD

보스가 아닌 몬스터/NPC용 머리 위 체력/체간/Life UI를 WidgetComponent 기반으로 구현했습니다.

Weapon Niagara

Trail, Aura, Jigen, BloodTrail 이펙트를 Notify에서 선택해 켜고 끄도록 공통화했습니다.

Foot Placement IK

플레이어, 보스, NPC의 경사면 접지를 위해 AnimInstance에서 FootPlacementAlpha를 상황별로 제어합니다.

Appendix C. Gameplay Details

전투 완성도와 플레이 감각을 보강한 세부 시스템

Hit Stop / Combat Feedback

피격, 공격 성공, 처형 상황에 따라 HitStop, SlowMotion, CameraShake, SFX/VFX를 분리해 전투 피드백을 조정했습니다.

Physical Hit Reaction

라이트/슈퍼아머 피격에서 몽타주 위에 PhysicalAnimation을 얹어 타격감을 보강합니다.

Jump Attack / Air Reaction

공중 피격, 공중 데스, 점프 패링, 점프 공격을 별도 리액션과 입력 흐름으로 분리했습니다.

Boss Parry Cycle

보스의 일반/완벽 패리 확률, 공방 탈출, 카운터, 공격권 전환 흐름을 별도 로직으로 관리합니다.

Facing / Turn

공격, 회피, 피격, 턴 몽타주별로 코드 Facing과 Notify 기반 Facing을 분리해 회전 틱을 줄였습니다.

Attack Trace Overrides

몽타주 Notify에서 Trace End 연장/축소, Radius, SampleCount, 패링/가드 불가 여부를 설정합니다.

Arrow / Projectile

화살과 번개 검기처럼 Trace가 아닌 투사체 공격도 같은 HitRequest 흐름으로 연결합니다.

Death / Respawn

Fake Death, Real Death, 공중 Death, 부활, 리스폰, 카메라/화면 페이드/락온 복구를 처리합니다.

Packaging Polish

SFX Preload, Niagara warmup 정리, 해상도 설정, Pause/Restart fade, Map Notice UI를 패키징 기준으로 다듬었습니다.

Part 2. DirectX Projects

엔진 기능을 직접 구현하며 쌓은 로우레벨 시스템 경험

DirectX Implementation Focus

- DirectX11 팀 프로젝트에서는 전투 카메라, 각성 컷신, 미니맵 UI처럼 플레이 경험을 보조하는 시스템을 구현했습니다.
- DirectX11 개인 프로젝트에서는 KeyFrame 기반 패링, OBB 무기 충돌, Point Instancing 파티클을 직접 구성했습니다.
- DirectX9 팀 프로젝트에서는 Layer 단위 TimeDelta 제어로 Time Stop 스킬을 구현했습니다.



Demon Slayer

DirectX gameplay system sample



Thymesia

DirectX gameplay system sample



IRA

DirectX gameplay system sample

DX 01. Combat Camera System

Demon Slayer Project - 전투축 기반 카메라 전환

Problem

플레이어가 적을 관통하거나 콤보 상황이 바뀔 때 카메라 축이 급격히 뒤집히는 문제가 있었습니다.

Solution

Player-Enemy 전투축과 BattleCenter를 기준으로 카메라 방향을 계산하고, 상황별 Battle/Side Camera 상태를 분리했습니다.

Result

전투 대상 가시성을 유지하면서 콤보 상황에서는 Side View로 공격 흐름을 강조했습니다.

[Demon Slayer Project] - Combat Camera System

Feature

전투 상황에 따라 카메라 시점을 동적으로 전환하는 전투 카메라 시스템

How it Works

플레이어-적 전투축을 기반으로 카메라 방향 계산

플레이어-적 중심(BattleCenter)을 바라보도록 시선 처리

Combo & 근거리 전투 상황에서만 Side View로 전환

Lerp 기반 보간으로 카메라 움직임 안정화

Result

Player-Enemy 중심 프레이밍을 사용해 전투 대상 가시성 확보

Combo 상황에서 Side Camera로 전환하여 공격 흐름 강조

카메라 이동과 시선 방향을 Lerp로 보간해 화면 안정성 유지



DX 02. Data-Driven Cutscene Camera

Demon Slayer Project - 각성 연출 컷 시퀀스

Problem

캐릭터별 각성 연출을 하드코딩하면 컷 추가와 수정이 어렵고 카메라 연산 코드가 복잡해졌습니다.

Solution

CutInCamDesc에 각도, 거리, 오프셋, 지속시간을 정의하고 단일 컷 연산과 시퀀스 제어를 분리했습니다.

Result

데이터 조합만으로 캐릭터별 컷신을 확장하고, 종료 후 전투 카메라로 복귀할 수 있게 했습니다.

[Demon Slayer Project] - Awakening Cutscene System

Feature

각성 연출을 다중 카메라 컷 시퀀스로 구성
카메라 연산과 컷 시퀀스 제어를 분리한 구조

How it Works

컷 데이터(CutInCamDesc)에 각도 / 거리 / 오프셋 / 지속시간 등을 정의
CutInFinish() → 단일 컷 카메라 연산
CutInCamera() → 컷 시퀀스 재생 및 전환

Result

데이터 기반 컷 구성으로 캐릭터별 연출을 코드 수정 없이 확장 가능
컷신 종료 후 전투 카메라로 자연스럽게 복귀

```
public:
    enum CUT_IN_FINISH
    { TANJIRO_AWAKE, ZENITSU_AWAKE, AKAZA_AWAKE,
      KYOGAI_AWAKE, TANJIRO_EXECUTION, END_FINISH };
```

→ 캐릭터별 enum



```
typedef struct Cut_In_Camera_Desc
{
    _bool bTarget_Is_Player = {};
    _bool bLookTarget_Is_Player = {};
    _float Angle_Hori = {};
    _float Angle_Verti = {};
    _float fDistance = {};
    _float fLifeTime = {};
    _bool bIs_Lerp = {};
    _vector vOffset = {};
    _vector vLookOffset = {};
}CutInCamDesc;

private:
    vector<CutInCamDesc> m_Cut_In_Finish[END_FINISH];
```

→ 컷 데이터 구조체

→ 캐릭터별 컷 시퀀스를 저장하는 vector 배열

DX 03. Minimap UV Mapping

Demon Slayer Project - 월드 좌표 기반 미니맵

Problem

맵마다 월드 크기가 달라 플레이어 위치를 UI 좌표로 일관되게 표시하기 어려웠습니다.

Solution

월드 좌표를 맵별 기준 범위로 0-1 UV에 정규화하고, 셰이더에서 표시 범위 밖을 Alpha Cut 처리했습니다.

Result

플레이어 주변 영역만 보여주는 동적 미니맵으로 탐색 가독성을 높였습니다.

World Position → UV Mapping

Concept

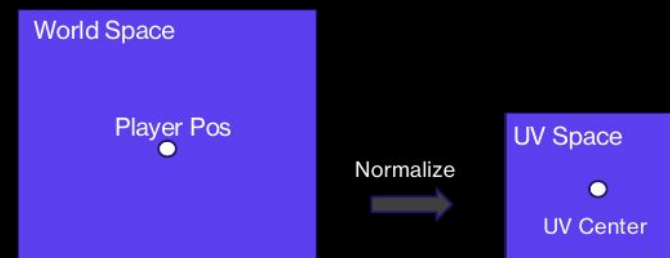
플레이어의 월드 좌표를 맵 기준 UV 좌표로 변환

How it Works

플레이어 월드 좌표 (x, z) 추출

맵별 월드 범위를 기준으로 **0-1 UV** 좌표로 정규화

계산된 UV 값을 미니맵 중심 좌표로 사용



```
if (m_Map_Type == 1) {
    // player Pos
    _vector Pos = m_pTargetTransformCom->Get_State(CTransform::STATE_POSITION);

    // World Position → UV coordinate
    m_UV_Centor_X = XMVectorGetX(Pos);
    m_UV_Centor_X = (m_UV_Centor_X - 35) / (95 - 29);

    m_UV_Centor_Y = XMVectorGetZ(Pos);
    m_UV_Centor_Y = (m_UV_Centor_Y - 1) / (79 - 1);

    // Player icon position
    m_UV_Player_X = m_UV_Centor_X;
    m_UV_Player_Y = m_UV_Centor_Y;
}
```

월드 좌표 범위를 기준으로 플레이어 위치를 0~1 UV 좌표로 정규화해 미니맵 중심 좌표를 계산
맵마다 월드 크기가 다르기 때문에, 각 레벨에 맞는 좌표 범위를 기준으로 UV 변환을 수행

Demon Slayer Project - Minimap System (1/2)

DX 04. KeyFrame Parry / OBB Hit

Thymesia Project - 타이밍 패링과 무기 충돌

Problem

공격 애니메이션마다 패링 가능 타이밍과 무기 충돌 판정이 달라 단순 충돌만으로는 전투 감각을 맞추기 어려웠습니다.

Solution

보스 공격 KeyFrame과 충돌 상태를 함께 검사하고, 무기 Bone Transform 기반 OBB를 SAT로 판정했습니다.

Result

공격별 패링 타이밍과 회전 무기 궤적이 반영된 근접 히트 판정을 구현했습니다.

[Thymesia Project] - Weapon Hit Detection (OBB Collision)

Feature

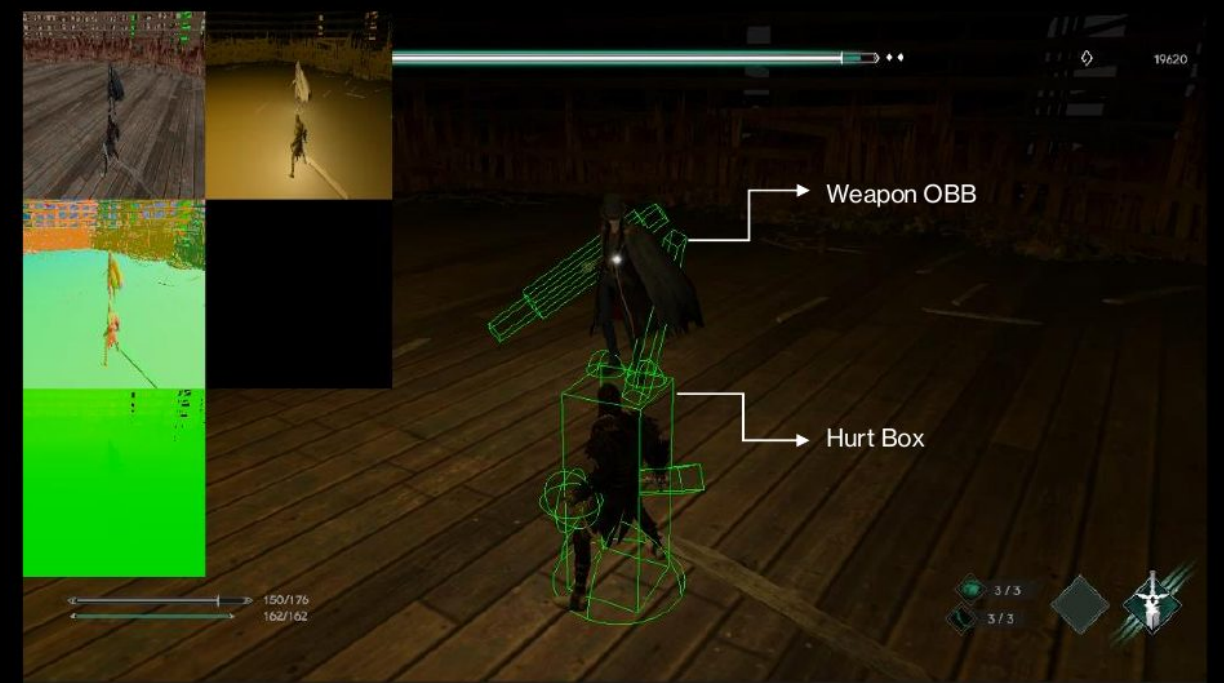
회전하는 무기 궤적을 반영하기 위해
OBB 기반 충돌 판정으로 근접 공격 히트 검출 구현

How it Works

무기의 Bone Transform을 기준으로 **Weapon OBB** 생성
매 프레임 월드 행렬을 적용해 회전된 충돌 박스 갱신
공격 애니메이션의 **특정 KeyFrame 구간**에서만 히트 판정 활성화
Weapon OBB와 Hurt Box 간 충돌 여부를 검사해 공격 성공 처리

Result

무기 방향과 자세 변화가 반영된 공격 판정 구현
단순 축정렬 박스보다 자연스러운 근접 무기 충돌 처리
공격 모션과 실제 히트 타이밍의 일관성 확보



DX 05. Particle Instancing

Thymesia Project - 공용 Point Instance Buffer

Problem

Parry, Spark, Dust 이펙트를 개별 오브젝트로 생성하면 반복 생성 비용과 관리 비용이 커졌습니다.

Solution

위치, 방향, 크기, 수명 정보를 InstanceDesc로 만들고 공용 Point Instance Buffer에서 일괄 갱신/렌더링했습니다.

Result

반복 파티클을 데이터 기반으로 관리하고 DrawIndexedInstanced로 렌더링 비용을 줄였습니다.

[Thymesia Project] - Particle Instancing

Feature

Parry, Spark, Dust 이펙트를

Point Instancing 기반 공용 버퍼 구조로 처리하는 경량 파티클 시스템 구현

How it Works

이펙트 발생 시 개별 오브젝트 생성 대신 Point Instance Data 생성

위치, 방향, 크기, 수명 정보를 **공용 인스턴싱 버퍼**에 등록

Tick에서 각 인스턴스의 이동, 확산, 수명 감소를 갱신

동일한 점 기반 파티클을 **Instancing** 방식으로 일괄 렌더링

Result

반복 생성되는 점 기반 파티클을 **경량 데이터** 기반으로 관리

Parry, Spark, Dust 이펙트를 하나의 공용 구조로 통합

Draw Call을 줄이고 렌더링 비용을 최소화



(Parry/Spark/Dust)_Particle Event

InstanceDesc 설정

Point Instance Buffer 등록

Tick Update

Instanced Rendering

DX 06. Time Stop Skill

IRA Project - Layer 단위 시간 제어

Problem

시간 정지 스킬에서 모든 오브젝트를 멈추면 플레이어와 스킬 예외 객체까지 같이 멈추는 문제가 생겼습니다.

Solution

Layer가 관리하는 오브젝트에 0.f TimeDelta를 전달하고, 예외 객체는 실제 TimeDelta를 참조하도록 분리했습니다.

Result

적, 투사체, 일반 오브젝트만 정지시키고 플레이어와 스킬 흐름은 유지하는 특수 전투 스킬을 구현했습니다.

[IRA Project] - Time Stop Skill System

Feature

Layer 단위 시간 제어를 통해 주변 오브젝트의 업데이트를 정지시키는 **Time Stop** Skill 구현

How it Works

시간 정지 시 레이어가 관리하는 오브젝트에 0.f의 TimeDelta를 전달
적, 일반 오브젝트, 투사체 등의 Tick을 사실상 정지 상태로 처리

실제 TimeDelta는 레이어에 별도로 저장

플레이어와 관련된 **예외 객체**는 실제 TimeDelta를 참조해 계속 갱신

Result

Time Stop 상태에서 주변 오브젝트의 움직임을 일괄 정지

플레이어와 스킬 관련 객체만 선택적으로 동작하도록 제어

전투 흐름을 일시적으로 제어하는 특수 스킬 구현

```
int CLayer::Update_Layer(const _float & fTimeDelta)
{
    int iResult = 0;

    // 실제 TimeDelta 저장
    // Time Stop 예외 대상(플레이어/화살/스킬 이펙트 등)이 참조 가능하도록 유지
    m_fTimeDelta = fTimeDelta;

    for (auto iter = m_uMapObject.begin(); iter != m_uMapObject.end(); )
    {
        if (m_fTimeStop == false) {
            // 일반 업데이트
            iResult = iter->second->Update_GameObject(fTimeDelta);
        }
        else {
            // Time Stop 시 TimeDelta를 0으로 전달해 업데이트 정지
            iResult = iter->second->Update_GameObject(0.f);
        }
        ++iter;
    }
}
```

```
int CPlayer::Update_GameObject(const _float& fTimeDelta)
{
    // Time Stop 예외 처리용 실제 TimeDelta 참조
    CLayer* pGameLogicLayer = Engine::Get_Layer(L"Layer_GameLogic");

    // 레이어에 저장된 실제 TimeDelta를 사용해 플레이어 업데이트
    _float Time = pGameLogicLayer->Get_TimeDelta();
}
```

IRA Project - Time Stop Skill System (1/1)